MPI 矩阵运算实验手册

版本 1.0

华为技术有限公司

目录

1 课程介绍 2
1.1 简介 2
1.2 内容描述 2
1.3 读者知识背景 2
1.4 实验环境说明 2
2 MPI 矩阵运算实验 3
2.1 实验介绍 3
2.1.1 关于本实验 3
2.1.2 教学目标 3
2.2 实验任务操作指导 3
2.2.1 创建矩阵乘法源码 3
2.2.2 创建卷积操作源码 8
2.2.3 创建池化操作源码 13
2.2.4 创建 makefile 16
2.2.5 进行编译 17
2.2.6 建立主机配置文件 18
2.2.7 运行监测 18
2.3 思考题及答案 25
3 缩略语表 26

1 课程介绍

1.1 简介

本书适用于学习并行计算课程的学生进行实验练习，完成本实验手册后，您将能更加充分理解集群 MPI 的矩阵乘法，以及基于此实现深度学习中的池化操作和卷积操作，并掌握在华为鲲鹏云服务器上如何运行实验。

1.2 内容描述

本实验指导书通过在华为鲲鹏云服务器上，编译运行矩阵乘法、池化、卷积等程序。完成实验操作后，读者会掌握简单的程序编写以及编译运行，集群 MPI 并行计算的配置以及加深对并行计算的了解。

1.3 读者知识背景

本课程为并行计算基础课程，为了更好地掌握本书内容，阅读本书的读者应首先具备以下基本条件：

- 具备基本的 Linux 命令能力；

1.4 实验环境说明

- 华为鲲鹏云主机、openEuler 20.03 操作系统；
- 安装 mpich-3.3.2.tar.gz；
- 安装 OpenBLAS-0.3.8.tar.gz；
- 每套实验环境可供 1 名学员上机操作。

2 MPI 矩阵运算实验

2.1 实验介绍

2.1.1 关于本实验

实现在多台主机上编译运行矩阵乘法、卷积、池化的程序。

2.1.2 教学目标

掌握多台主机矩阵乘法、池化、卷积等操作的程序编写及编译运行。

2.2 实验任务操作指导

2.2.1 创建矩阵乘法源码

实验说明：随机生成大小为 1024 *1024 的矩阵作为输入，实现对应的矩阵乘法，矩阵乘法主要利用了矩阵划分方法，每一个工作节点的进程负责某一行和某一列的乘法，主节点则负责矩阵的划分以及分发到各个工作节点。

以下步骤均在 ecs-00-0001 上，以 zhangsan 用户执行。

执行以下命令 ,创建 matrix 目录存放该程序的所有文件，并进入 matrix 目录（四台主机都执行）

```
mkdir /home/zhangsan/matrix
cd /home/zhangsan/matrix
```

执行以下命令，创建矩阵乘法源码 gemm.cpp（四台主机都执行）

```
vim gemm.cpp
```

代码内容如下：

```
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <iostream>
#include "mpi.h"
```

```
// #include <cblas.h>

using namespace std;

// void CheckStatus(MPI_Status &status) {
//         if (status.MPI_ERROR != MPI_SUCCESS) {
//                 cout << MPI::Get_error_class(status.MPI_ERROR);
//         }
// }

void randMat(int rows, int cols, float *&Mat) {
    Mat = new float[rows * cols];
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            Mat[i * cols + j] = 1.0;
}

void openmp_sgemm(int m, int n, int k, float *&leftMat, float *&rightMat,
                              float *&resultMat) {
    // rightMat is transposed
#pragma omp parallel for
    for (int row = 0; row < m; row++) {
        for (int col = 0; col < k; col++) {
            resultMat[row * k + col] = 0.0;
            for (int i = 0; i < n; i++) {
                resultMat[row * k + col] +=
                        leftMat[row * n + i] * rightMat[col * n + i];
            }
        }
    }
    return;
}

void blas_sgemm(int m, int n, int k, float *&leftMat, float *&rightMat,
                            float *&resultMat) {
    // cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasTrans, m, k, n, 1.0, leftMat,
    // n, rightMat, n, 0.0, resultMat, k);
}

void mpi_sgemm(int m, int n, int k, float *&leftMat, float *&rightMat,
                           float *&resultMat, int rank, int worldsize, bool blas) {
    int rowBlock = sqrt(worldsize);
    if (rowBlock * rowBlock > worldsize)
        rowBlock -= 1;
    int colBlock = rowBlock;

    int rowStride = m / rowBlock;
    int colStride = k / colBlock;
```

```
worldsize = rowBlock * colBlock; // we abandom some processes.
// so best set process to a square number.

float *res;

if (rank == 0) {
    float *buf = new float[k * n];
    // transpose right Mat
    for (int r = 0; r < n; r++) {
        for (int c = 0; c < k; c++) {
            buf[c * n + r] = rightMat[r * k + c];
        }
    }

    for (int r = 0; r < k; r++) {
        for (int c = 0; c < n; c++) {
            rightMat[r * n + c] = buf[r * n + c];
        }
    }

    delete buf;

    MPI_Request sendRequest[2 * worldsize];
    MPI_Status status[2 * worldsize];
    for (int rowB = 0; rowB < rowBlock; rowB++) {
        for (int colB = 0; colB < colBlock; colB++) {
            rowStride = (rowB == rowBlock - 1) ? m - (rowBlock - 1) * (m / rowBlock)
                                                : m / rowBlock;
            colStride = (colB == colBlock - 1) ? k - (colBlock - 1) * (k / colBlock)
                                                : k / colBlock;
            int sendto = rowB * colBlock + colB;
            if (sendto == 0)
                continue;
            MPI_Isend(&leftMat[rowB * (m / rowBlock) * n], rowStride * n, MPI_FLOAT,
                      sendto, 0, MPI_COMM_WORLD, &sendRequest[sendto]);
            MPI_Isend(&rightMat[colB * (k / colBlock) * n], colStride * n,
                      MPI_FLOAT, sendto, 1, MPI_COMM_WORLD,
                      &sendRequest[sendto + worldsize]);
        }
    }
    for (int rowB = 0; rowB < rowBlock; rowB++) {
        for (int colB = 0; colB < colBlock; colB++) {
            int recvfrom = rowB * colBlock + colB;
            if (recvfrom == 0)
                continue;
            MPI_Wait(&sendRequest[recvfrom], &status[recvfrom]);
            MPI_Wait(&sendRequest[recvfrom + worldsize],
```

```
                    &status[recvfrom + worldsize]);
        }
    }
    res = new float[(m / rowBlock) * (k / colBlock)];
} else {
    if (rank < worldsize) {
        MPI_Status status[2];
        rowStride = ((rank / colBlock) == rowBlock - 1)
                            ? m - (rowBlock - 1) * (m / rowBlock)
                            : m / rowBlock;
        colStride = ((rank % colBlock) == colBlock - 1)
                            ? k - (colBlock - 1) * (k / colBlock)
                            : k / colBlock;
        if (rank != 0) {
            leftMat = new float[rowStride * n];
            rightMat = new float[colStride * n];
        }
        if (rank != 0) {
            MPI_Recv(leftMat, rowStride * n, MPI_FLOAT, 0, 0, MPI_COMM_WORLD,
                    &status[0]);
            MPI_Recv(rightMat, colStride * n, MPI_FLOAT, 0, 1, MPI_COMM_WORLD,
                    &status[1]);
        }
        res = new float[rowStride * colStride];
    }
}
MPI_Barrier(MPI_COMM_WORLD);

if (rank < worldsize) {
    rowStride = ((rank / colBlock) == rowBlock - 1)
                        ? m - (rowBlock - 1) * (m / rowBlock)
                        : m / rowBlock;
    colStride = ((rank % colBlock) == colBlock - 1)
                        ? k - (colBlock - 1) * (k / colBlock)
                        : k / colBlock;
    if (!blas)
        openmp_sgemm(rowStride, n, colStride, leftMat, rightMat, res);
    else
        blas_sgemm(rowStride, n, colStride, leftMat, rightMat, res);
}
MPI_Barrier(MPI_COMM_WORLD);

if (rank == 0) {
    MPI_Status status;
    float *buf = new float[(m - (rowBlock - 1) * (m / rowBlock)) *
                                    (k - (colBlock - 1) * (k / colBlock))];
    float *temp_res;
    for (int rowB = 0; rowB < rowBlock; rowB++) {
```

```
        for (int colB = 0; colB < colBlock; colB++) {
            rowStride = (rowB == rowBlock - 1) ? m - (rowBlock - 1) * (m / rowBlock)
                                               : m / rowBlock;
            colStride = (colB == colBlock - 1) ? k - (colBlock - 1) * (k / colBlock)
                                               : k / colBlock;
            int recvfrom = rowB * colBlock + colB;
            if (recvfrom != 0) {
                temp_res = buf;
                MPI_Recv(temp_res, rowStride * colStride, MPI_FLOAT, recvfrom, 0,
                        MPI_COMM_WORLD, &status);
            } else {
                temp_res = res;
            }
            for (int r = 0; r < rowStride; r++)
                for (int c = 0; c < colStride; c++)
                    resultMat[rowB * (m / rowBlock) * k + colB * (k / colBlock) +
                            r * k + c] = temp_res[r * colStride + c];
        }
    }
  } else {
    rowStride = ((rank / colBlock) == rowBlock - 1)
                        ? m - (rowBlock - 1) * (m / rowBlock)
                        : m / rowBlock;
    colStride = ((rank % colBlock) == colBlock - 1)
                        ? k - (colBlock - 1) * (k / colBlock)
                        : k / colBlock;
    if (rank < worldsize)
        MPI_Send(res, rowStride * colStride, MPI_FLOAT, 0, 0, MPI_COMM_WORLD);
  }
  MPI_Barrier(MPI_COMM_WORLD);

  return;
}

int main(int argc, char *argv[]) {
  if (argc != 5) {
    cout << "Usage: " << argv[0] << " M N K use-blas\n";
    exit(-1);
  }

  int rank;
  int worldSize;
  MPI_Init(&argc, &argv);

  MPI_Comm_size(MPI_COMM_WORLD, &worldSize);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);

  int m = atoi(argv[1]);
```

```
    int n = atoi(argv[2]);
    int k = atoi(argv[3]);
    int blas = atoi(argv[4]);

    float *leftMat, *rightMat, *resMat;

    struct timeval start, stop;
    if (rank == 0) {
        randMat(m, n, leftMat);
        randMat(n, k, rightMat);
        randMat(m, k, resMat);
    }
    gettimeofday(&start, NULL);
    mpi_sgemm(m, n, k, leftMat, rightMat, resMat, rank, worldSize, blas);
    gettimeofday(&stop, NULL);
    if (rank == 0) {
        cout << "mpi matmul: "
                << (stop.tv_sec - start.tv_sec) * 1000.0 +
                        (stop.tv_usec - start.tv_usec) / 1000.0
                << " ms" << endl;

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < k; j++)
                if (int(resMat[i * k + j]) != n) {
                    cout << resMat[i * k + j] << "error\n";
                    exit(-1);
                }
            // cout << resMat[i * k + j] << ' ';
            // cout << endl;
        }
    }
    MPI_Finalize();
}
```

2.2.2 创建卷积操作源码

实验说明：实现卷积计算操作，卷积核的大小为 4*4，卷积算法种类很多，在这里我们主要使用 img2col 算法来加速卷积算法，img2col 算法原理为利用数据的重排布把卷积转化为矩阵乘法。

以下步骤均在 ecs-oo-0001 上，以 zhangsan 用户执行。

执行以下命令，进入 matrix 目录（四台主机都执行）

```
cd /home/zhangsan/matrix
```

执行以下命令，创建卷积操作源码 conv.cpp（四台主机都执行）

```
vim conv.cpp
```

代码内容如下：

```
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <string.h>

#include <iostream>

#include "mpi.h"

#include <cblas.h>
#include <assert.h>

using namespace std;

void randMat(int rows, int cols, float *&Mat) {
	Mat = new float[rows * cols];
	for (int i = 0; i < rows; i++)
		for (int j = 0; j < cols; j++) Mat[i * cols + j] = 1.0;
}

int get_steps(int kernel, int step, int len) {
	if (kernel > len) return 0;
	return (len - kernel) / step + 1;
}


inline void img2col_conv_kernel(int leftAnchorX, int leftAnchorY, int rightAnchorX,
                                int rightAnchorY, const int xKernel, const int yKernel, const int xStep,
                                const int yStep, float *&img, float *&kernel, float *&conv) {

	int imgRows = rightAnchorX - leftAnchorX,
		imgCols = rightAnchorY - leftAnchorY;

	int convRows = get_steps(xKernel, xStep, imgRows);
	int convCols = get_steps(yKernel, yStep, imgCols);

	float *flattenImg = new float[convRows * convCols * xKernel * yKernel];

// #pragma omp parallel for
//	for (int i = leftAnchorX; i < rightAnchorX - xKernel; i += xStep) {
//		for (int r = 0; r < xKernel; r++) {
//			for (int j = leftAnchorY; j < rightAnchorY - yKernel; j += yStep) {
//				int pos = (i - leftAnchorX)/xStep * convCols + (j - leftAnchorY) / yStep;
```

```
//                          memcpy(&flattenImg[pos * xKernel * yKernel + r * yKernel], &img[(i + r) * imgCols + j],
sizeof(float) * yKernel);
//                      }
//              }
//          }

// #pragma omp parallel for
      // for (int i = 0; i < convRows * convCols; i++) {
      //         conv[i] = 0.0;
      //         for (int j = 0; j < xKernel * yKernel; j++) {
      //                 conv[i] += flattenImg[i*xKernel*yKernel + j] * kernel[j];
      //         }
      // }

      cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans, convRows * convCols, 1, xKernel * yKernel, 1.0,
flattenImg, xKernel * yKernel, kernel, 1, 0.0, conv, 1);
      // cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasTrans, convRows * convCols, 1, xKernel * yKernel, 1.0,
flattenImg, xKernel * yKernel, kernel, convRows * convCols, 0.0, conv, 1);

      delete flattenImg;

}

inline void naive_conv_kernel(int leftAnchorX, int leftAnchorY, int rightAnchorX,
                              int rightAnchorY, const int xKernel, const int yKernel, const int xStep,
                              const int yStep, float *&img, float *&kernel, float *&conv) {
      int imgRows = rightAnchorX - leftAnchorX,
          imgCols = rightAnchorY - leftAnchorY;

      int convRows = get_steps(xKernel, xStep, imgRows);
      int convCols = get_steps(yKernel, yStep, imgCols);
#pragma omp parallel for
      for (int i = leftAnchorX; i < rightAnchorX - xKernel; i += xStep) {
            for (int j = leftAnchorY; j < rightAnchorY - yKernel; j += yStep) {
                  // to[i / xStride * to_n + j] = 0.0;
                  int pos = (i - leftAnchorX)/xStep * convCols + (j - leftAnchorY) / yStep;
                  conv[pos] = 0.0;
                  for (int r = i; r < i + xKernel; r++)
                        for (int c = j; c < j + yKernel; c++) {
                              conv[pos] += img[r * imgCols + c] *
                                           kernel[(r - i) * yKernel + (c - j)];
                        }
            }
      }
}

void mpi_convolution(int m, int n, int xKernel, int yKernel, int xStep,
                     int yStep, float *&img, float *&kernel, float *&conv,
```

```
                        int rank, int worldsize, bool img2col) {
const int total_xsteps = get_steps(xKernel, xStep, m);
const int total_ysteps = get_steps(yKernel, yStep, n);
const int xsteps_per_proc = total_xsteps / worldsize;
const int last_xsteps = total_xsteps - xsteps_per_proc * (worldsize - 1);
int steps;
if (rank == 0) {
    MPI_Request *sendRequest = new MPI_Request[worldsize];
    MPI_Status *status = new MPI_Status[worldsize];
    for (int i = 1; i < worldsize; i++) {
        steps = (i == worldsize - 1) ? last_xsteps : xsteps_per_proc;
        MPI_Isend(&img[i * xsteps_per_proc * xStep * n],
                  (steps * xStep + xKernel - xStep) * n, MPI_FLOAT, i, 0,
                  MPI_COMM_WORLD, &sendRequest[i]);
    }
    for (int i = 1; i < worldsize; i++) {
        MPI_Wait(&sendRequest[i], &status[i]);
    }
    delete sendRequest;
    delete status;
} else {
    MPI_Status status;
    steps = (rank == worldsize - 1) ? last_xsteps : xsteps_per_proc;
    img = new float[(steps * xStep + xKernel - xStep) * n];
    MPI_Recv(img, (steps * xStep + xKernel - xStep) * n, MPI_FLOAT, 0, 0,
             MPI_COMM_WORLD, &status);
    conv = new float[steps * total_ysteps];
}
MPI_Barrier(MPI_COMM_WORLD);

steps = (rank == worldsize - 1) ? last_xsteps : xsteps_per_proc;

if (img2col)
    img2col_conv_kernel(0, 0, steps * xStep + xKernel - xStep, n, xKernel, yKernel,
                        xStep, yStep, img, kernel, conv);
else
    naive_conv_kernel(0, 0, steps * xStep + xKernel - xStep, n, xKernel, yKernel,
                        xStep, yStep, img, kernel, conv);

MPI_Barrier(MPI_COMM_WORLD);

if (rank == 0) {
    MPI_Status status;
    for (int i = 1; i < worldsize; i++) {
        steps = (i == worldsize - 1) ? last_xsteps : xsteps_per_proc;
        MPI_Recv(&conv[i * xsteps_per_proc * total_ysteps],
                  steps * total_ysteps, MPI_FLOAT, i, 0, MPI_COMM_WORLD,
                  &status);
```

```
            }
        } else {
            MPI_Send(conv, steps * total_ysteps, MPI_FLOAT, 0, 0, MPI_COMM_WORLD);
        }
        MPI_Barrier(MPI_COMM_WORLD);

        return;
}

int main(int argc, char *argv[]) {
        if (argc != 4) {
            cout << "Usage: " << argv[0] << " M N enabled-img2col";
            exit(-1);
        }

        int rank;
        int worldSize;
        MPI_Init(&argc, &argv);

        MPI_Comm_size(MPI_COMM_WORLD, &worldSize);
        MPI_Comm_rank(MPI_COMM_WORLD, &rank);

        int m = atoi(argv[1]);
        int n = atoi(argv[2]);
        int img2col = atoi(argv[3]);

        int xKernel = 3, yKernel = 3;
        int xStep = 1, yStep = 1;

        float *Img, *Conv;

        struct timeval start, stop;
        if (rank == 0) {
            randMat(m, n, Img);
            randMat(get_steps(xKernel, xStep, m), get_steps(yKernel, yStep, n),
                    Conv);
        }

        float *Kernel = new float[xKernel*yKernel];
        for (int i = 0; i < xKernel*yKernel; i++) Kernel[i] = 1.0;

        gettimeofday(&start, NULL);
        mpi_convolution(m, n, xKernel, yKernel, xStep, yStep, Img, Kernel, Conv,
                        rank, worldSize, img2col);
        gettimeofday(&stop, NULL);

        if (rank == 0) {
            cout << "mpi convolution: "
```

```
                << (stop.tv_sec - start.tv_sec) * 1000.0 +
                        (stop.tv_usec - start.tv_usec) / 1000.0
                << " ms" << endl;

        for (int i = 0; i < min(10, m); i++) {
            for (int j = 0; j < min(10, n); j++)
                cout << Conv[i * n + j] << ' ';
            cout << endl;
        }
    }
    delete Img;
    delete Conv;
    MPI_Finalize();
}
```

2.2.3 创建池化操作源码

实验说明：基于乘法的程序实现池化计算操作，池化使用的 kernel 大小为 4*4，池化操作与卷积操作类似，更为简单，只需取每一个感受野内的最大值。

以下步骤均在 ecs-oo-0001 上，以 zhangsan 用户执行。

执行以下命令，进入 matrix 目录（四台主机都执行）

```
cd /home/zhangsan/matrix
```

执行以下命令，创建卷积操作源码 pooling.cpp（四台主机都执行）

```
vim pooling.cpp
```

代码内容如下：

```
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>

#include <iostream>

#include "mpi.h"

using namespace std;

void randMat(int rows, int cols, float *&Mat) {
    Mat = new float[rows * cols];
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++) Mat[i * cols + j] = 1.0;
}

inline void pooling_kernel(int leftAnchorX, int leftAnchorY, int rightAnchorX,
```

```
                                    int rightAnchorY, int xStride, int yStride,
                                    int from_m, int from_n, int to_m, int to_n,
                                    float *&from, float *&to) {
    if ((rightAnchorX - leftAnchorX) % xStride != 0 ||
        (rightAnchorY - leftAnchorY) % yStride != 0)
        exit(-1);
    if (leftAnchorX % xStride != 0 || leftAnchorY % yStride != 0) exit(-2);

    for (int i = leftAnchorX; i < rightAnchorX; i += xStride) {
        for (int j = leftAnchorY; j < rightAnchorY; j += yStride) {
            // to[i / xStride * to_n + j] = 0.0;
            float temp = to[i / xStride * to_n + j / yStride];
            for (int r = i; r < i + xStride; r++)
                for (int c = j; c < j + yStride; c++) {
                    temp = max(temp, from[r * from_n + c]);
                }
            to[i / xStride * to_n + j / yStride] = temp;
        }
    }
}

void mpi_pooling(int m, int n, int xstride, int ystride, float *&mat,
                 float *&res, int rank, int worldsize) {
    if (m % xstride || n % ystride) {
        cout << "matrix size and stride do not match \n";
        return;
    }
    const int xstrides_per_proc = (m / xstride) / worldsize;
    int strides;
    if (rank == 0) {
        MPI_Request *sendRequest = new MPI_Request[worldsize];
        MPI_Status *status = new MPI_Status[worldsize];
        for (int i = 1; i < worldsize; i++) {
            strides = (i < worldsize - 1)
                          ? xstrides_per_proc
                          : (m / xstride) - xstrides_per_proc * (worldsize - 1);
            MPI_Isend(&mat[i * xstrides_per_proc * xstride * n],
                      strides * xstride * n, MPI_FLOAT, i, 0, MPI_COMM_WORLD,
                      &sendRequest[i]);
        }
        for (int i = 1; i < worldsize; i++) {
            MPI_Wait(&sendRequest[i], &status[i]);
        }
        delete sendRequest;
        delete status;
    } else {
        MPI_Status status;
        strides = (rank < worldsize - 1)
```

```
                                ? xstrides_per_proc
                                : (m / xstride) - xstrides_per_proc * (worldsize - 1);
            mat = new float[strides * xstride * n];
            MPI_Recv(mat, strides * xstride * n, MPI_FLOAT, 0, 0, MPI_COMM_WORLD,
                    &status);
            res = new float[strides * (n / ystride)];
        }
        MPI_Barrier(MPI_COMM_WORLD);

        strides = (rank < worldsize - 1)
                        ? xstrides_per_proc
                        : (m / xstride) - xstrides_per_proc * (worldsize - 1);

        // pooling_kernel(rank * xstrides_per_proc * xstride, 0,
        //                 (rank * xstrides_per_proc + strides) * xstride, n,
        //                 xstride, ystride, m, n, strides, n / ystride, mat, res);

        pooling_kernel(0, 0, strides * xstride, n, xstride, ystride,
                        strides * xstride, n, strides, n / ystride, mat, res);

        MPI_Barrier(MPI_COMM_WORLD);

        if (rank == 0) {
            MPI_Status status;
            for (int i = 1; i < worldsize; i++) {
                strides = (i < worldsize - 1)
                                ? xstrides_per_proc
                                : (m / xstride) - xstrides_per_proc * (worldsize - 1);
                MPI_Recv(&res[i * xstrides_per_proc * (n / ystride)],
                        strides * (n / ystride), MPI_FLOAT, i, 0, MPI_COMM_WORLD,
                        &status);
            }
        } else {
            MPI_Send(res, strides * (n / ystride), MPI_FLOAT, 0, 0, MPI_COMM_WORLD);
        }
        MPI_Barrier(MPI_COMM_WORLD);

        return;
}

int main(int argc, char *argv[]) {
        if (argc != 3) {
            cout << "Usage: " << argv[0] << " M N";
            exit(-1);
        }

        int rank;
        int worldSize;
```

```
    MPI_Init(&argc, &argv);

    MPI_Comm_size(MPI_COMM_WORLD, &worldSize);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int m = atoi(argv[1]);
    int n = atoi(argv[2]);

    int xstride = 4, ystride = 4;

    float *Mat, *resMat;

    struct timeval start, stop;
    if (rank == 0) {
        randMat(m, n, Mat);
        randMat(m / xstride, n / ystride, resMat);
    }
    gettimeofday(&start, NULL);
    mpi_pooling(m, n, xstride, ystride, Mat, resMat, rank, worldSize);
    gettimeofday(&stop, NULL);
    if (rank == 0) {
        cout << "mpi pooling: "
            << (stop.tv_sec - start.tv_sec) * 1000 * 1000L +
                (stop.tv_usec - start.tv_usec)
            << endl;

        // for (int i = 0; i < min(10, m); i++) {
        //     for (int j = 0; j < min(10, n); j++)
        //         cout << Mat[i * k + j] << ' ';
        //     cout << endl;
        // }
    }
    delete Mat;
    delete resMat;
    MPI_Finalize();
}
```

2.2.4 创建 makefile

执行以下命令，创建 Makefile（四台主机都执行）

```
vim Makefile
```

代码内容如下：

```
CC = mpic++
CCFLAGS = -O2 -fopenmp
LDFLAGS = -lopenblas
```

```
all:    gemm conv pooling

gemm: gemm.cpp
    ${CC} ${CCFLAGS} gemm.cpp -o gemm ${LDFLAGS}

conv: conv.cpp
    ${CC} ${CCFLAGS} conv.cpp -o conv ${LDFLAGS}

pooling: pooling.cpp
    ${CC} ${CCFLAGS} pooling.cpp -o pooling ${LDFLAGS}

clean:
    rm gemm conv pooling
```

```
CC = mpic++
CCFLAGS = -O2 -fopenmp
LDFLAGS = -lopenblas

all: gemm conv pooling

gemm: gemm.cpp
        ${CC} ${CCFLAGS} gemm.cpp -o gemm ${LDFLAGS}

conv: conv.cpp
        ${CC} ${CCFLAGS} conv.cpp -o conv ${LDFLAGS}

pooling: pooling.cpp
        ${CC} ${CCFLAGS} pooling.cpp -o pooling ${LDFLAGS}

clean:
        rm gemm conv  pooling
```

2.2.5 进行编译

执行以下命令，获取包（四台主机都执行）

```
wget https://github.com/xianyi/OpenBLAS/archive/v0.3.8.tar.gz
```

执行以下命令，进行编译（四台主机都执行）

```
tar -zxvf v0.3.8.tar.gz && cd OpenBLAS-0.3.8
make -j2
sudo make PREFIX=/usr/local/openblas install
sudo chmod -R 777 /usr/local/openblas/
```

执行以下命令，完成安装（四台主机都执行）

```
sudo ln -s /usr/local/openblas/lib/libopenblas.so /usr/lib/libopenblas.so
```

执行以下命令，配置 OpenBLAS 环境（四台主机都执行）

```
vim ~/.bashrc
```

添加下列内容

```
export CPLUS_INCLUDE_PATH=$CPLUS_INCLUDE_PATH:/usr/local/openblas/include
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/openblas/lib
```

```
# Source default setting
[ -f /etc/bashrc ] && . /etc/bashrc

# User environment PATH
PATH="$HOME/.local/bin:$HOME/bin:$PATH"
export PATH
export CPLUS_INCLUDE_PATH=$CPLUS_INCLUDE_PATH:/usr/local/openblas/include

export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/openblas/lib
```

执行以下命令，使环境变量均生效（四台主机都执行）

```
source ~/.bashrc
```

执行以下命令，进行编译（四台主机都执行）

```
make
```

```
[zhangsan@ecs-hw-0001 matrix]$ make
mpic++ -O2 -fopenmp gemm.cpp -o gemm -lopenblas
mpic++ -O2 -fopenmp conv.cpp -o conv -lopenblas
mpic++ -O2 -fopenmp pooling.cpp -o pooling -lopenblas
[zhangsan@ecs-hw-0001 matrix]$ ll
total 104K
-rwx------ 1 zhangsan zhangsan  72K Jul 21 13:17 conv
-rw------- 1 zhangsan zhangsan 7.1K Jul 20 15:06 conv.cpp
-rwx------ 1 zhangsan zhangsan  72K Jul 21 13:17 gemm
-rw------- 1 zhangsan zhangsan 7.5K Jul 20 15:06 gemm.cpp
-rw------- 1 zhangsan zhangsan   60 Jul 20 15:53 hostfile
-rw------- 1 zhangsan zhangsan  327 Jul 20 16:04 Makefile
-rwx------ 1 zhangsan zhangsan  71K Jul 21 13:17 pooling
-rw------- 1 zhangsan zhangsan 5.1K Jul 20 15:06 pooling.cpp
```

2.2.6 建立主机配置文件

执行以下命令，建立主机配置文件（四台主机都执行）

```
vim /home/zhangsan/matrix/hostfile
```

添加内容如下：

```
ecs-hw-0001:2
ecs-hw-0002:2
ecs-hw-0003:2
ecs-hw-0004:2
```

```
[zhangsan@ecs-hw-0001 matrix]$ more hostfile
ecs-hw-0001:2
ecs-hw-0002:2
ecs-hw-0003:2
ecs-hw-0004:2
[zhangsan@ecs-hw-0001 matrix]$
```

2.2.7 运行监测

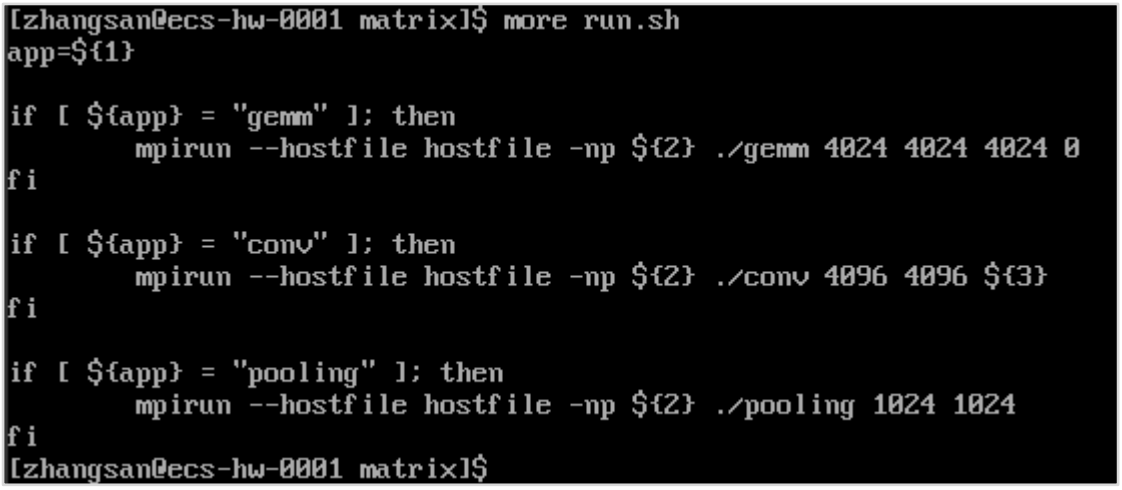
编写 run.sh 脚本，内容如下：

```
app=${1}

if [ ${app} = "gemm" ]; then
    mpirun --hostfile hostfile -np ${2} ./gemm 4024 4024 4024 0
fi

if [ ${app} = "conv" ]; then
    mpirun --hostfile hostfile -np ${2} ./conv 4096 4096 ${3}
fi

if [ ${app} = "pooling" ]; then
    mpirun --hostfile hostfile -np ${2} ./pooling 1024 1024
fi
```



① 分别执行以下命令，查看矩阵乘法运行结果（只需要在 ecs-hw-0001 上执行）

```
bash run.sh gemm 1
bash run.sh gemm 2
bash run.sh gemm 3
bash run.sh gemm 4
bash run.sh gemm 5
bash run.sh gemm 6
bash run.sh gemm 7
bash run.sh gemm 8
```

1-8 数字表示启动处理的进程数量。

结果如下：

```
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh gemm 1
mpi matmul: 55671.3 ms
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh gemm 2
mpi matmul: 133995 ms
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh gemm 3

Authorized users only. All activities may be monitored and reported.
mpi matmul: 134081 ms
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh gemm 4

Authorized users only. All activities may be monitored and reported.
mpi matmul: 34077.7 ms
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh gemm 5

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi matmul: 33934.4 ms
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh gemm 6

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi matmul: 34249.8 ms
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh gemm 7

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi matmul: 34357.5 ms
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh gemm 8

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi matmul: 34037.3 ms
[zhangsan@ecs-hw-0001 matrix]$
```

通过上述运行 ,从整体大致可以看出 ,随着进程数量的增加 ,由进程数量为 1 时候耗时 55671.3ms 到进程数量为 8 耗时 34037.3ms，耗时减少。

① 分别执行以下命令，查看卷积运行结果（只需要在 ecs-hw-0001 上执行）

```
bash run.sh conv 1 0
bash run.sh conv 2 0
bash run.sh conv 3 0
bash run.sh conv 4 0
bash run.sh conv 5 0
bash run.sh conv 6 0
bash run.sh conv 7 0
bash run.sh conv 8 0

bash run.sh conv 1 1
bash run.sh conv 2 1
bash run.sh conv 3 1
bash run.sh conv 4 1
bash run.sh conv 5 1
bash run.sh conv 6 1
```

```
bash run.sh conv 7 1
bash run.sh conv 8 1
```

conv 后面的 1-8 数字表示启动处理的进程数量。最后面的 0 表示 vanilla convolution kernel，1 表示 img2col kernel。

结果如下：

```
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 1 0
mpi convolution: 405.142 ms
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 2 0
mpi convolution: 4831.52 ms
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 3 0

Authorized users only. All activities may be monitored and reported.
mpi convolution: 3277.17 ms
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
[zhangsan@ecs-hw-0001 matrix]$
```

```
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 4 0

Authorized users only. All activities may be monitored and reported.
mpi convolution: 2588.25 ms
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9
[zhangsan@ecs-hw-0001 matrix]$
```

```
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 5 0

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi convolution: 2098.45 ms
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 6 0

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi convolution: 1975.35 ms
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
[zhangsan@ecs-hw-0001 matrix]$
```

```
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 7 0

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi convolution: 1538.91 ms
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 8 0

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi convolution: 1499.49 ms
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
9 9 9 9 9 9 9 9 9 9
[zhangsan@ecs-hw-0001 matrix]$
```

通过上述运行，从整体大致可以看出，随着进程数量的增加，耗时减少。

```
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 1 1
mpi convolution: 261.146 ms
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 2 1
mpi convolution: 4399.42 ms
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 3 1

Authorized users only. All activities may be monitored and reported.
mpi convolution: 2968.95 ms
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
[zhangsan@ecs-hw-0001 matrix]$ _
```

```
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 4 1

Authorized users only. All activities may be monitored and reported.
mpi convolution: 2479.61 ms
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 5 1

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi convolution: 2134.49 ms
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
[zhangsan@ecs-hw-0001 matrix]$ _
```

```
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 6 1

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi convolution: 1987.48 ms
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 7 1

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi convolution: 1650.62 ms
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
[zhangsan@ecs-hw-0001 matrix]$
```

```
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh conv 8 1

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi convolution: 1387.53 ms
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
[zhangsan@ecs-hw-0001 matrix]$
```

通过上述运行，从整体大致可以看出，随着进程数量的增加，耗时减少。

② 分别执行以下命令，查看池化运行结果（只需要在 ecs-hw-0001 上执行）

```
bash run.sh pooling 1
bash run.sh pooling 2
bash run.sh pooling 3
bash run.sh pooling 4
bash run.sh pooling 5
bash run.sh pooling 6
bash run.sh pooling 7
```

```
bash run.sh pooling 8
```

1-8 数字表示启动处理的进程数量。

结果如下：

```
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh pooling 1
mpi pooling: 2270
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh pooling 2
mpi pooling: 202912
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh pooling 3

Authorized users only. All activities may be monitored and reported.
mpi pooling: 162924
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh pooling 4

Authorized users only. All activities may be monitored and reported.
mpi pooling: 142939
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh pooling 5

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi pooling: 145849
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh pooling 6

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi pooling: 122620
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh pooling 7

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi pooling: 158937
[zhangsan@ecs-hw-0001 matrix]$ bash run.sh pooling 8

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.

Authorized users only. All activities may be monitored and reported.
mpi pooling: 128493
[zhangsan@ecs-hw-0001 matrix]$
```

通过上述运行，从整体大致可以看出，随着进程数量的增加，耗时减少。

2.3 思考题及答案

- 如何添加 C、C+头文件以及库路径加入环境变量？

说明：《高性能与并行计算》课程配套实验手册中的实验内容由上海交通大学计算机科学与工程系吴晨涛老师提供，华为公司负责实验手册文档的编写。

3 缩略语表

缩略语	英文全称	中文全称
ECS	Elastic Cloud Server	弹性云服务器
MPI	MessagePassingInterface	信息传递接口